Disk Stacking

AlgoExpert · Hard · Dynamic Programming

The problem

- Disks [width, depth, height]; stack them so each disk is strictly smaller in all three dimensions than the one below, maximising total height.
- Returns the chosen disks ordered from top (smallest) to bottom (largest).

Intuition

- Sort disks by height; each disk's best stack is itself on top of the best compatible stack below.

Approach

- 1. Sort by height
- 2. For each disk, check smaller earlier disks
- 3. Keep the best height and a predecessor link
- 4. Rebuild the stack

Java solution

```java
public class DiskStacking {
    public static List<Integer[]> diskStacking(List<Integer[]> disks) {
        List<Integer[]> d = new ArrayList<>(disks);
        d.sort(Comparator.comparingInt(x -> x[2]));
        int n = d.size(), best = 0;
        int[] h = new int[n], prev = new int[n];
        for (int i = 0; i < n; i++) {
            h[i] = d.get(i)[2];
            prev[i] = -1;
            for (int j = 0; j < i; j++) {
                Integer[] a = d.get(j), b = d.get(i);
                if (a[0] < b[0] && a[1] < b[1] && a[2] < b[2] && h[j] + b[2] > h[i]) { h[i] = h[j] + b[2]; prev[i] = j; }
            }
            if (h[i] > h[best]) best = i;
        }
        LinkedList<Integer[]> out = new LinkedList<>();
        for (int i = best; i != -1; i = prev[i]) out.addFirst(d.get(i));
        return out;
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Recap

- Sort disks by height; each disk's best stack is itself on top of the best compatible stack below.
- Complexity: O(n^2)